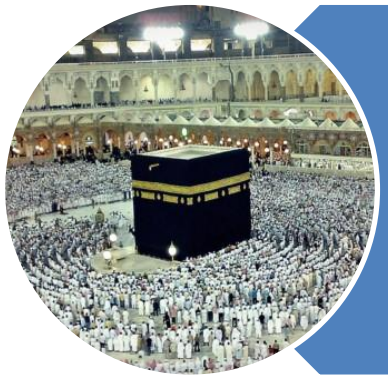
Data Structures - I

CPCS 204

رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي اَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

CPCS 204

STACKS

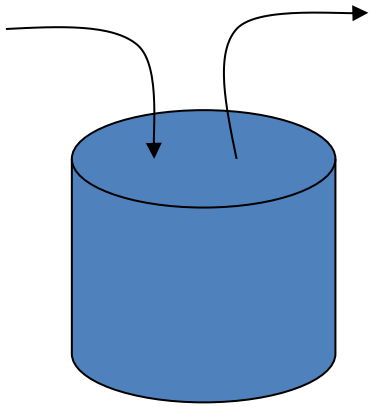
Implementation

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Implementation - Introduction

- Implementation
  - Arrays
  - Linked List
- Accessibility
  - Top, LIFO
- Operations
  - Push
  - Pop
  - isEmpty
  - isFull
  - top



CPCS 204

Stack Implementation

Arrays

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Array Implementation

- Array
  - Insertion
    - first, last, Anywhere
  - Deletion
    - first, last, Anywhere
  - Traversal
  - Searching
  - Sorting
  - Merging

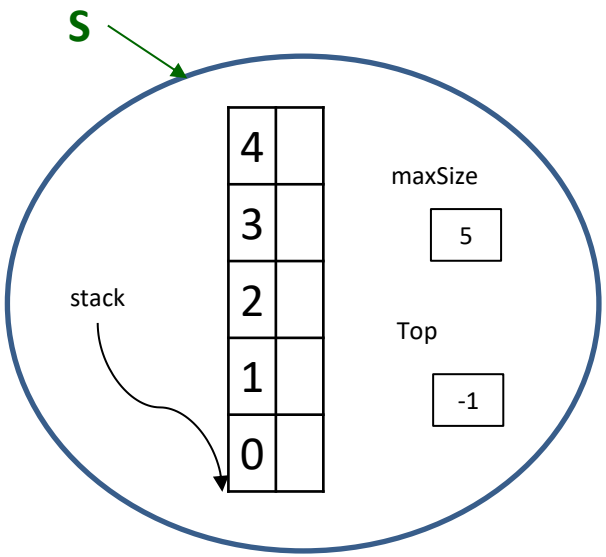
- Array as STACK
  - Push
    - Only at Top
  - Pop
    - Only from Top
  - isEmpty
    - Top == -1
  - isFull
    - Top == maxSize-1
  - top or peek
    - Return the value at Top

CPCS 204

Stack Array - Class

```
public class StackArray {
   private int[] stack;
   private int top;
   private int maxSize;

   public StackArray(int size) {
      maxSize = size;
      stack = new int[maxSize];
      top = -1;
   }

}
```

```
StackArray S;
S = new StackArray (5);
```



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Stack Array - Methods

- `public boolean isEmpty()`
- `public boolean isFull()`
- `public void push(int j)`
- `public int pop()`
- `public int top()`

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Stack Methods - `isEmpty()`

```
public class StackArray {
   private int[] stack;
   private int top;
   private int maxSize;

   public StackArray(int size) {
      maxSize = size;
      stack = new int[maxSize];
      top = -1;
   }

}
```

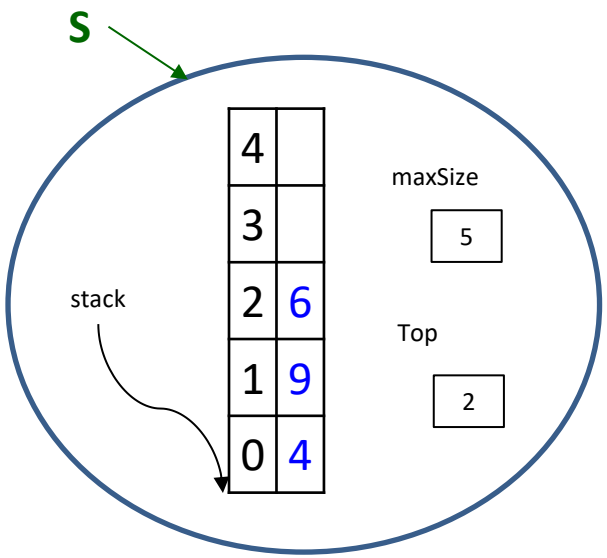
```
public boolean isEmpty() {
    if (top == -1)
        return true;
    else
        return false;
}
```

```
public boolean isEmpty() {
    return (top == -1);
}
```

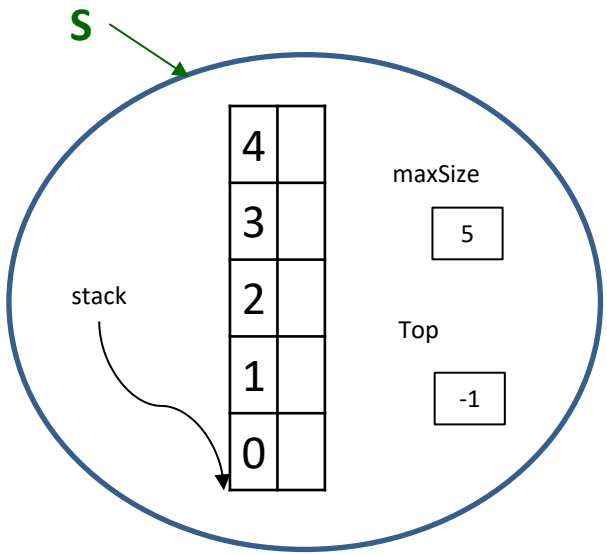
CPCS 204

Stack Methods - `isEmpty()`

```
public boolean isEmpty() {
    return (top == -1);
}
```



False



True

Stack Methods - `isFull()`

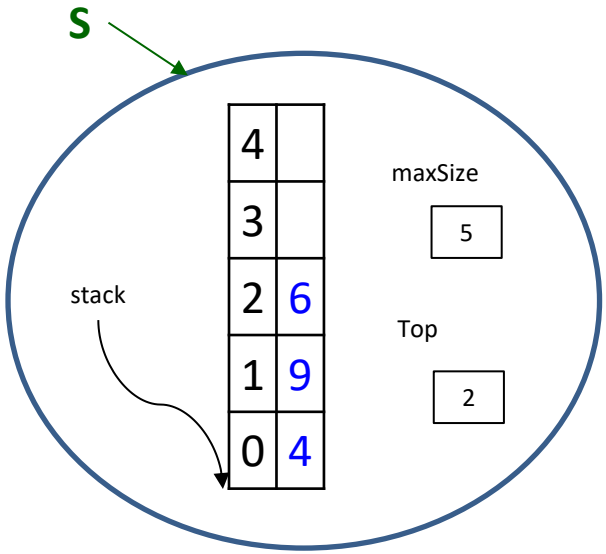
```
public class StackArray {
   private int[] stack;
   private int top;
   private int maxSize;

   public StackArray(int size) {
      maxSize = size;
      stack = new int[maxSize];
      top = -1;
   }

}
```

```
public boolean isFull() {
    if (top == maxSize-1)
        return true;
    else
        return false;
}
```

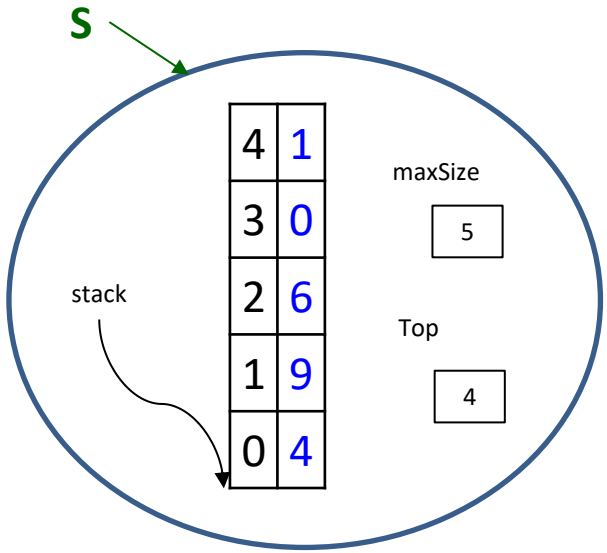
```
public boolean isFull() {
    return (top == maxSize-1);
}
```

Stack Methods - `isFull()`

```
public boolean isFull() {
    return (top == maxSize-1);
}
```



False



True

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Stack Methods - `push(value)`

- Remember, we can only push if the stack is not full
  - Meaning, if there is room to push
- So if the stack is full
  - We print an error message.
    - Or throw an Exception, showing the push could not be done
- If there is room
  - We increment top to point to the next space in the stack
  - Then we simply copy the value into this new top position

Stack Methods - **push** (value)

```
public void push(int value) {
    if (isFull())
        System.out.println("Cannot PUSH; stack is full.");
    else{
            top++;
            stack[top] = value;
     }
}
```

```
public void push(int value) {
    if (isFull())
        System.out.println("Cannot PUSH; stack is full.");
    else
        stack[++top] = value;
}
```

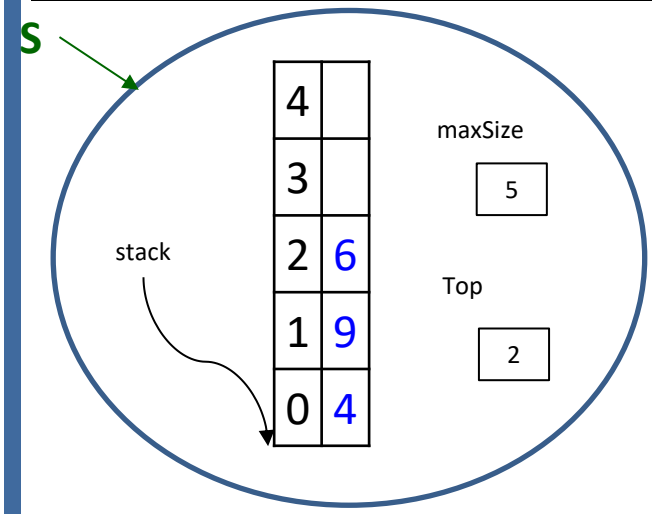

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

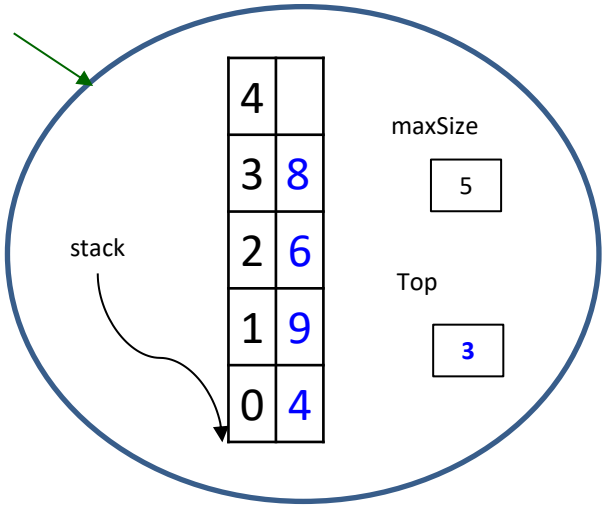
CPCS 204

Stack Methods - **push** (value)

```
public void push(int value) {
    if (isFull())
        System.out.println("Cannot PUSH; stack is full.");
    else
        stack[++top] = value;
}
```



Value =8



Dr. Jonathan (Yahya) Cazalas False

Dr. Muhammad Umair Ramzan

Stack Methods - `pop` ( )

- Remember, we can only pop if the stack is not empty
  - Meaning, there is at least one element to pop
- So if the stack is empty
  - We print an error message
    - Or throw an exception showing that we cannot pop
- If the stack has at least one element:
  - We return the value at position `top`
  - At the same time, we decrement `top`
  - This makes the `top` "pointer" refer to the next item down in the stack

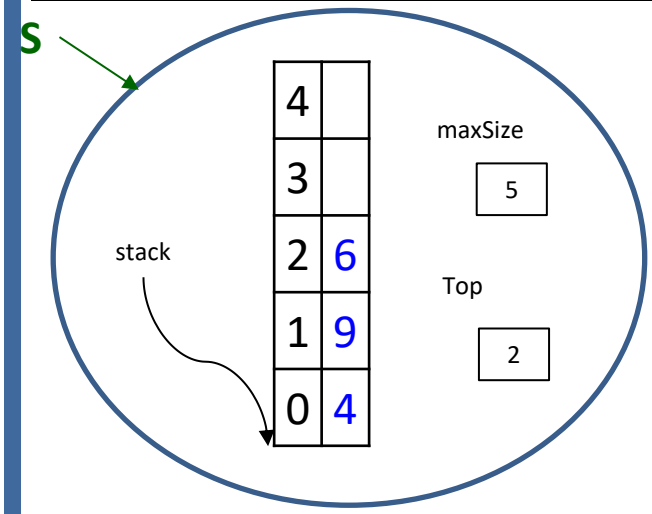
Stack Methods - pop ( )

```
public int pop() {
    int value;
    if (isEmpty())
        System.out.println("Cannot POP; stack is empty.");
    else{
            value = stack[top];
            top--;
            return value;
    }
}
```

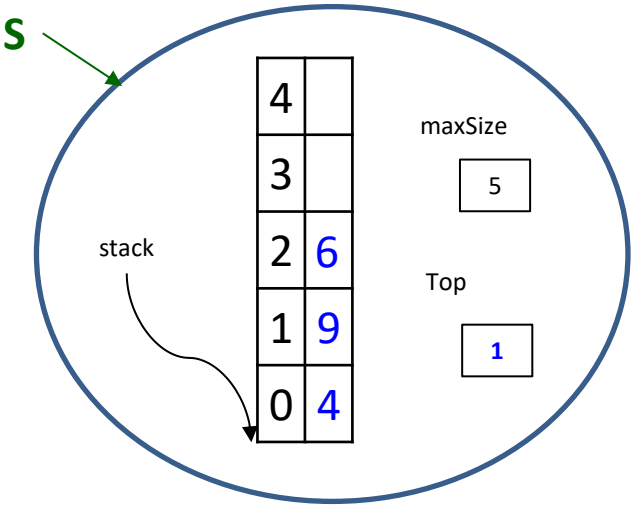
```
public int pop() {
    if (isEmpty())
        System.out.println("Cannot POP; stack is empty.");
    else
        return stack[top--];
}
```

Stack Methods - pop ( )

```
public int pop() {
    if (isEmpty())
        System.out.println("Cannot POP; stack is empty.");
    else
        return stack[top--];
}
```



Value =6



Dr. Jonathan (Yahya) Cazalas False

Dr. Muhammad Umair Ramzan

Stack Methods - `top()` or `peek()`

- The `top` method is very similar to pop
- Remember, we can only check for the top of the stack if the stack is not empty
  - Meaning, there is at least one element in the stack
- So if the stack is empty
  - We print an error message
- If the stack has at least one element:
  - We simply return the topmost element
  - But we do NOT decrement

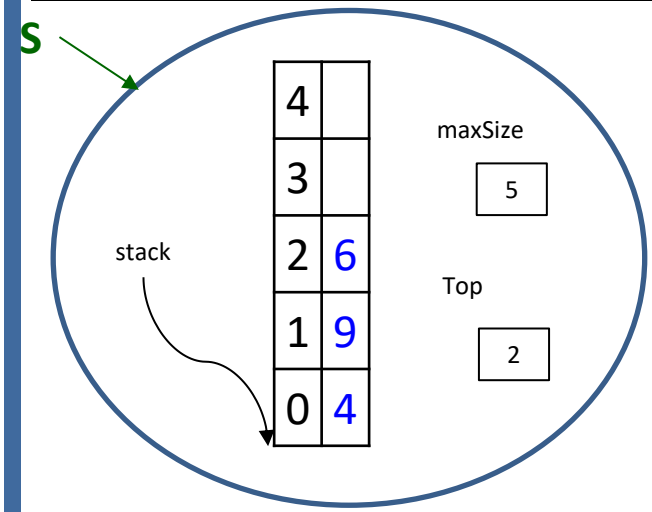
Stack Methods - top ( )

```
public int pop() {
    if (isEmpty())
        System.out.println("Cannot POP; stack is empty.");
    else
        return stack[top--];
}
```

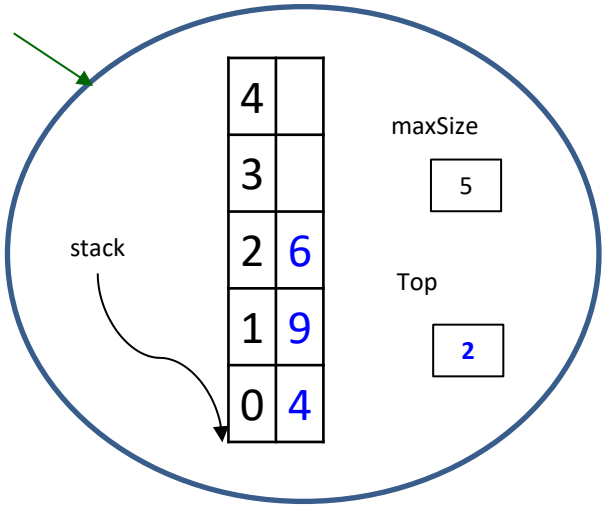
```
public int top() {
    if (isEmpty())
        System.out.println("Cannot TOP; stack is empty.");
    else
        return stack[top];
}
```

Stack Methods - top ( )

```
public int top() {
    if (isEmpty())
        System.out.println("Cannot TOP; stack is empty.");
    else
        return stack[top];
}
```



Value =6



 False

CPCS 204

Stack Implementation

Linked List

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Linked List Implementation

- Linked List
  - Insertion
    - first, last, Anywhere
  - Deletion
    - first, last, Anywhere
  - Traversal
  - Searching
  - Merging

- Linked List as STACK
  - Push
    - Only at Top
  - Pop
    - Only from Top
  - isEmpty
    - Top == -1
  - top or peek
    - Return the value at Top

Linked List Implementation

- Linked List as STACK
  - We essentially use a standard linked list
  - But we limit the functionality of a linked list
    - Thus creating the behavior required of a stack
  - Step 1:
    - Choose the location of top!
    - Where should we choose top to be? What is best?
      - Top could be the "front" of the list OR the "end" of the list
      - Which is better?
    - If we choose top to be the end, we must traverse to the end for every PUSH and every POP…a lot of traversing!
    - BEST choice: we let top be the front of our list.

Linked List Implementation

- Linked List as STACK
  - So we will create two classes (like Linked-Lists)
    - StackNode.java (this is like the LLnode class)
    - StackLL.java (this is like the LinkedList class)
  - We use most of the SAME methods as used for the Linked List class
  - However, we RESTRICT the functionality
    - Now, we cannot insert anywhere in the list
    - We can only insert at the top (head)
    - Now, we cannot delete from anywhere in the list
    - We can only delete from the top (head)

Stack Linked List – Node Class

```
public class StackNode {
    private int data;
    private StackNode next;

    public StackNode(int i) {
          data = i;
          next = null;
    }
    public StackNode(int i, StackNode n) {
          data = i;
          next = n;
    }
}
```

Stack Linked List – Stack LL Class

```
public class StackLL {
    private StackNode Top;

    public StackLL() {
          Top = null;
     }

    // POP, PUSH, TOP and more...
}
```


Dr. Jonathan (Yahya) Cazalas

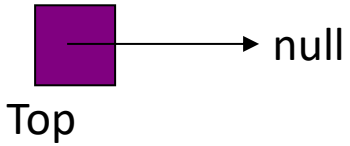
Dr. Muhammad Umair Ramzan

Stack Linked List - **Methods**

- `public boolean isEmpty()`
- `public void push(int j)`
- `public int pop()`
- `public int top()`

Stack Methods - `isEmpty()`

```
public boolean isEmpty() {
    return Top == null;
}
```



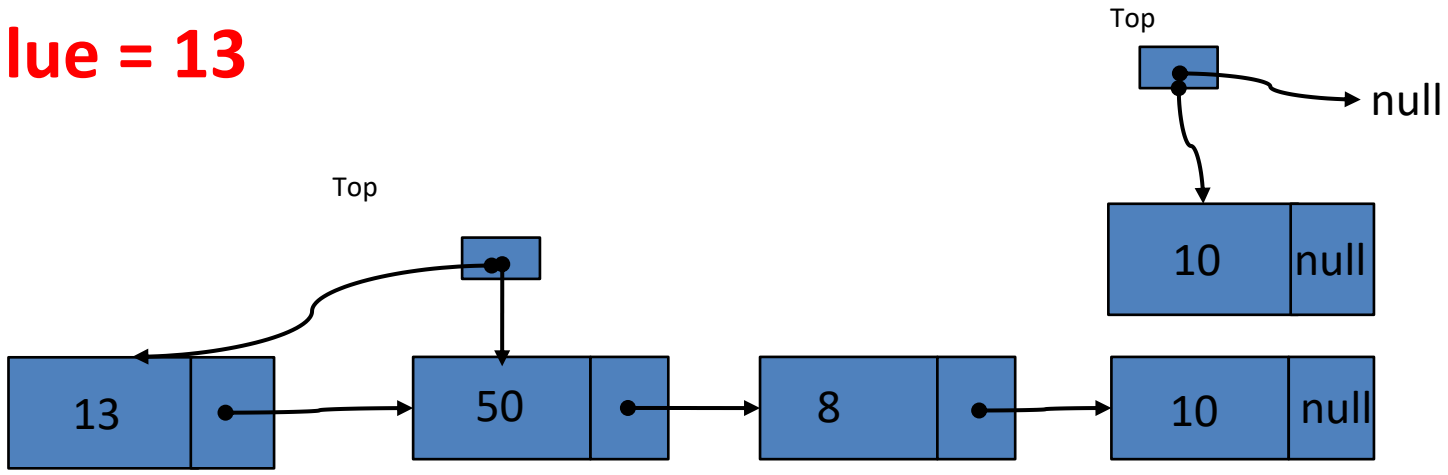
Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Stack Methods - **push** (value)

```
public void push(int data) {

    Top = new StackNode(data, Top);

}
```

Value = 13

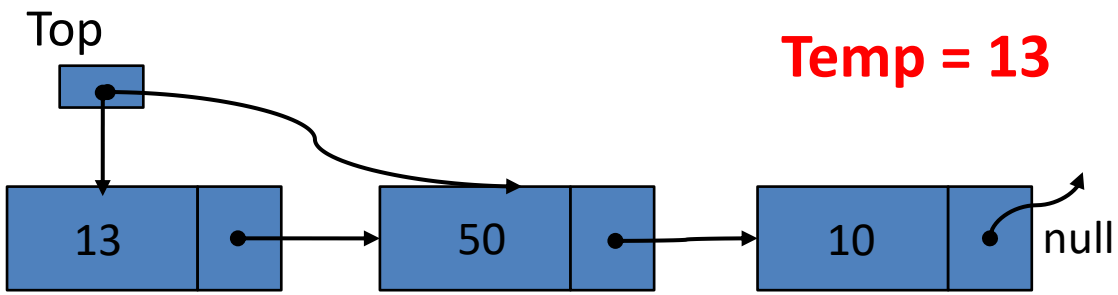


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Stack Methods - pop ( )

```
public int pop( ) {
    if (!isEmpty()) {
        int temp = Top . data;
        Top = Top . next;
        return temp;
    }
}
```

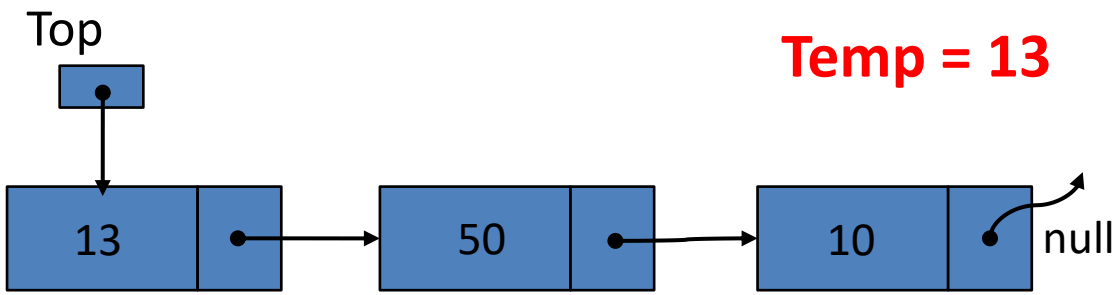


Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Stack Methods - top ( )

```
public int top( ) {
    if (!isEmpty()) {
        return Top.data;
    }
}
```



Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Review Questions
Stacks

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

MCQ

- Using an array-based implementation of a stack, what is the least efficient (the slowest) place for the top of the stack to be:
  a) At the end of the array
  b) At the start of the array
  c) At the middle of the array
  d) All are the same

MCQ

- Which of the following is not a characteristic of a stack?
  a) First In Last Out
  b) Last In First Out
  c) Insertion in the middle
  d) None of above

MCQ

- Which operation cannot be applied if the abstract data type is full?
  a) push
  b) pop
  c) dequeue
  d) peek

MCQ

- **Which of the following is not the property of stack?**
  a) First in, Last Out
  b) Last in, First Out
  c) Accessed by Top only
  d) May be implemented using Linked List
  e) None

MCQ

- **Which of the following "operation(s)" do not change the contents of the stack?**
  a) push
  b) pop
  c) top
  d) isEmpty

MCQ

- **While converting "infix" to "postfix", the stack is used to store**
  a) Operands
  b) Operators
  c) Postfix Expression
  d) 1st and 2nd both
  e) None

MCQ

- **While evaluating "postfix", the stack is used to store**
  a) Operands
  b) Results
  c) Postfix Expression
  d) 1st and 2nd both
  e) None

MCQ

- **Which of the following "operators" have the "highest" priority while "out of stack?**
  - a) + (plus)
  - b) - (minus)
  - c) * (Multiply)
  - d)) (closing paraenthesis)
  - e) All have same priority

MCQ

- **Consider the linked list implementation of a stack. Which of the following node is considered as Top of the stack?**
  - a) First node
  - b) Last node
  - c) Any node
  - d) Middle Node

MCQ

- If data is a circular array of **CAPACITY** elements, and **rear** is an index of that array, what is the formula to update the index **rear**?
  a) (rear % 1) + CAPACITY
  b) rear % (1 + CAPACITY)
  c) (rear + 1) % CAPACITY
  d) rear + (1 % CAPACITY)

MCQ

- When a Stack is implemented using an array; the **isFull()** method returns true if

a) top == -1
b) top == maxSize
c) top == maxSize – 1
d) top == null

MCQ

- What is the value of the following postfix expression after you evaluate it?

5 8 * 9 – 3 - 2 2 + /

a) 4
b) 7
c) 21
d) 28
e) None of the above

MCQ

- Suppose the rule of the party is that the participants who arrive later will leave earlier. Which data structure is appropriate to store the participants?
  a) Stack
  b) Queue
  c) Priority Queue
  d) Linked List

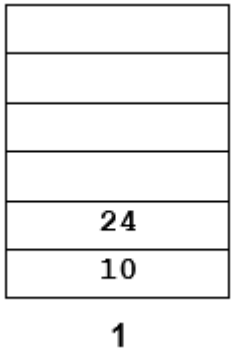
Postfix Evaluation

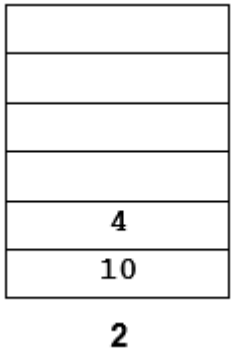
- **Evaluate** the following postfix expression using a stack. Additionally, you must show the contents of the stack at the indicated points (1, 2, and 3) in the postfix expression.

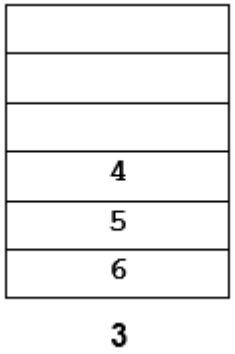
```
                  1                   2                   3
10  4  6  *   3  4  2  -  *  /   -  5  6  2  -   *  -
```

Postfix Evaluation

			1					2							3			
10	4	6	*		3	4	2	–	*	/		–	5	6	2	–	*	–







Final Answer: −14

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Postfix Evaluation

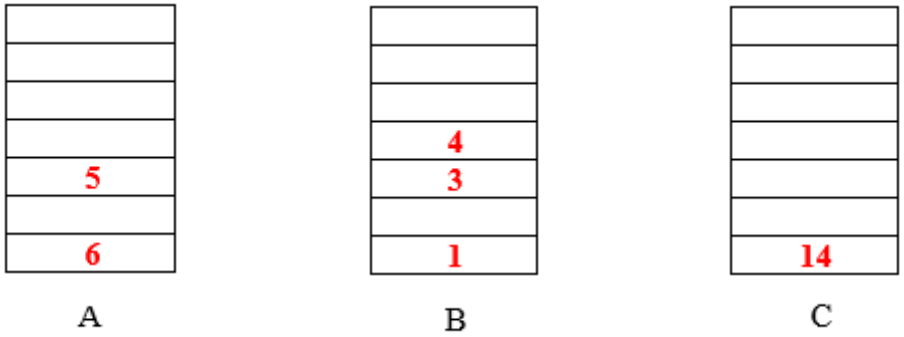
- **Evaluate** the following postfix expression using a stack. Additionally, you must show the status of the stack at given point A, B and C. Don't forget to write the final result of postfix expression.

$$6\ 2\ 3\ +\ \overset{A}{\ }\ -\ 3\ 8\ 2\ /\ \overset{B}{\ }\ +\ *\ 2\ *\ \overset{C}{\ }\ 3\ +$$

Postfix Evaluation

A B C

6 2 3 + - 3 8 2 / + * 2 * 3 +



Final Result = 17

Postfix Evaluation

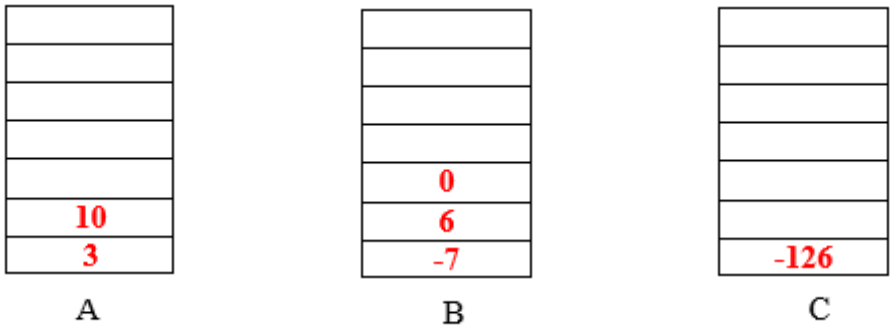
- **Evaluate** the following postfix expression using a stack. Additionally, you must show the status of the stack at given point A, B and C. Don't forget to write the final result of postfix expression.

3 8 2 + A - 6 2 3 / B + * 3 * C 2 +

Postfix Evaluation

A B C

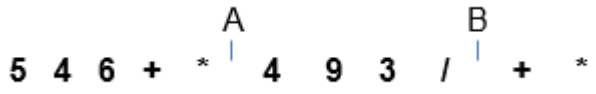
3 8 2 + - 6 2 3 / + * 3 * 2 +



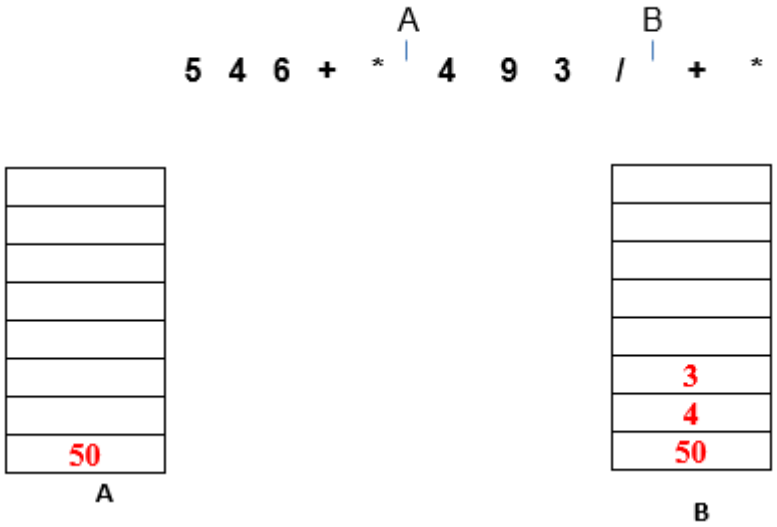
Final Value = -124

Postfix Evaluation

- **Evaluate** the following postfix expression using a stack. Additionally, you must show the status of the stack at given point **A** and **B**. Don't forget to write the final result of postfix expression.



Postfix Evaluation



Result is: 350

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

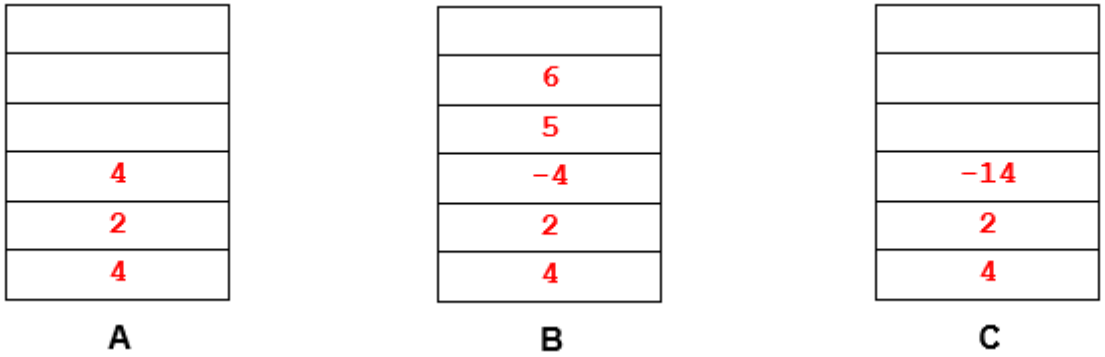
Postfix Evaluation

- **Evaluate** the following postfix expression using a stack. Additionally, you must show the contents of the stack at the indicated points (**A**, **B**, and **C**) in the postfix expression.

```
                A           B           C
4  12  6  /  4  8  -  5  6  4  -  *  -  -  +
```

Postfix Evaluation

				A				B				C			
4	12	6	/	4	8	–	5	6	4	–	*	–	–	+	



Final Answer: 20

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Infix to Postfix

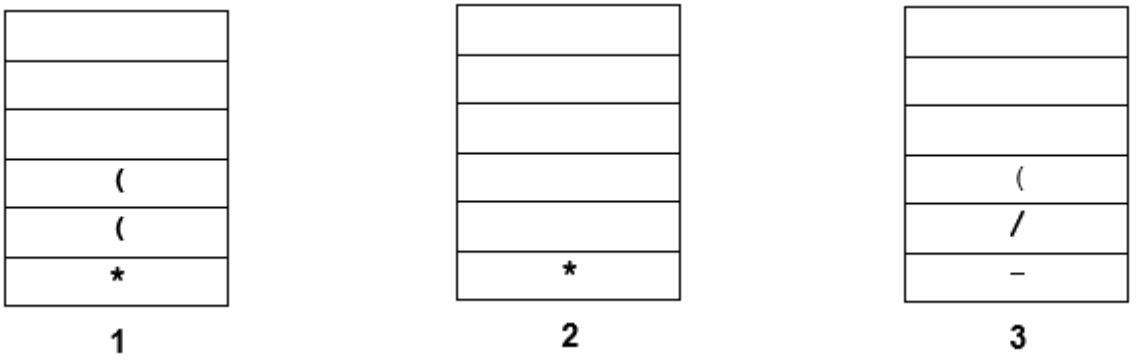
- **Convert** the following infix expression into its equivalent postfix expression using a stack. Additionally, you must show the contents of the stack at the indicated points (1, 2, and 3) in the infix expression.

```
                1           2                 3
5 * ( ( 2 + 3 ) * 4 ) - 6 * 10 / ( 3 + 2 )
```

Infix to Postfix

1 2 3

5 * ((2 + 3) * 4) - 6 * 10 / (3 + 2)



Postfix expression: 5 2 3 + 4 * * 6 10 * 3 2 + / -

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

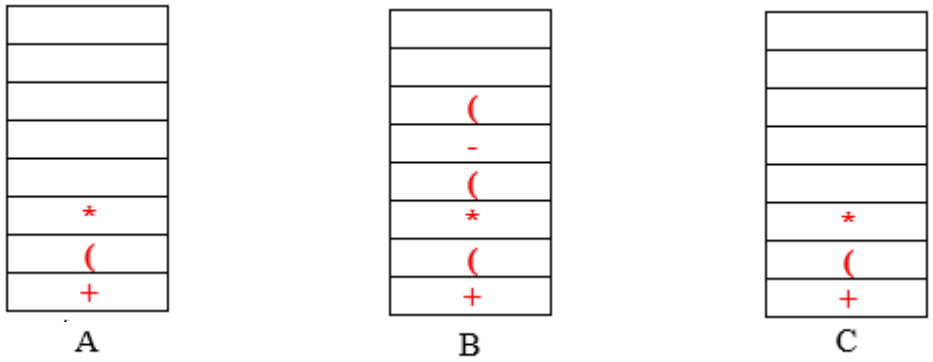
Infix to Postfix

- **Convert** the following infix expression into its equivalent postfix expression using a stack. Additionally, you must show the status of the stack at given point A, B and C. Don't forget to write the resultant postfix expression.

$$2 + (5 * \overset{A}{} (2 + 5 - (\overset{B}{} 4 / 3 + 1)) * \overset{C}{} 4) - 5$$

Infix to Postfix

$$2 + (5 * \overset{A}{(2 + 5} - (\overset{B}{4 / 3 + 1)) *} \overset{C}{4) - 5}$$



2 5 2 5 + 4 3 / 1 + - * 4 * + 5 -

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Infix to Postfix

- **Convert** the following infix expression into its equivalent postfix expression using a stack. Additionally, you must show the status of the stack at given point **X, Y** and **Z**. Don't forget to write the resultant postfix expression.

```
                X           Y           Z
( A + B ) * C ( D / ( J + D ) - K *F)
```

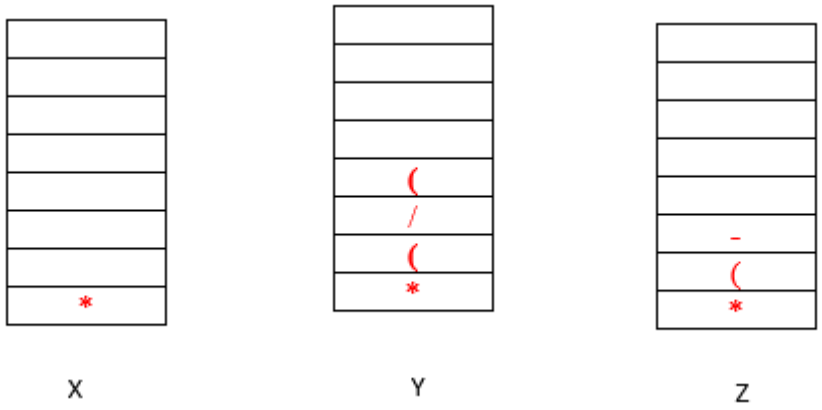

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan

Infix to Postfix

(A + B) * C (D / (J + D) - K * F)

X Y Z



A B + C D J D + / K F * - *

Dr. Jonathan (Yahya) Cazalas

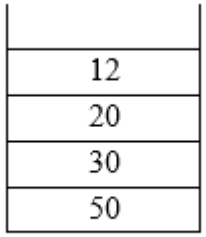
Dr. Muhammad Umair Ramzan

Stack Coding

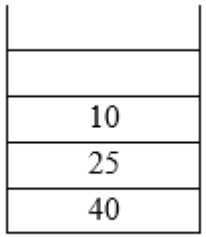
- Assume you are given 2 stacks as parameters, stacks A and B. Both of these stacks are sorted with the minimum value on the top. You must write a method that creates a new stack, Stack C, which contains all the values from Stack A and Stack B. Stack C must also be sorted. To be clear, the largest value from Stacks A and B must be at the bottom of Stack C, and the smallest value from Stacks A and B must be at the top of Stack C. You can assume that there are no duplicate values in Stacks A and B. You are only allowed to use the stack operations such as pop(), push(), top(), and size(). No other data structure such as arrays are allowed, but you may use additional Stacks if needed. Of course, you can assume that each node of the stack has a single "data" member, which is an int. This allows you to compare those integer values. Finally, you must return a reference of your new Stack C.

Stack Coding

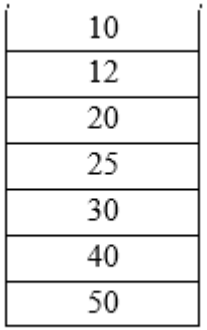
- Example



Stack A



Stack B



*New Stack C

CPCS 204

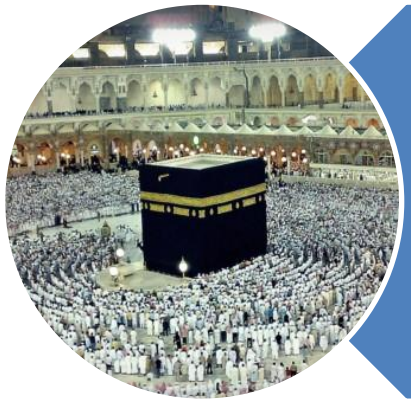
Stack Coding

```
public Stack mergeSortedStacks(Stack A, Stack B) {
     Stack C = new Stack();
     Stack D = new Stack();
     while (!A.isEmpty() && !B.isEmpty()) {
            if (A.top() < B.top)
               C.push(A.pop());
            else
               C.push(B.pop());
     }
     while (!A.isEmpty()) {
        C.push(A.pop());
     }
     while (!B.isEmpty()) {
        C.push(B.pop());
     }
     while (!C.isEmpty()) {
        D.push(C.pop());
     }
     C = D;
     return C;
}
```

Data Structures - I

"No Bonus" marks for this course anymore

CPCS 204



Allah (Alone) is Sufficient for us, and He is the Best Disposer of affairs (for us).**(Quran 3 : 173)**

Dr. Jonathan (Yahya) Cazalas

Dr. Muhammad Umair Ramzan